

# Group Mini-Project Report Template

## Cloud Computing Module — Mini-Project 2

|                       |                                         |
|-----------------------|-----------------------------------------|
| Project Title         | Cloud Computing Module - Mini Project 2 |
| Module                | <COMP3041J / Cloud Computing>           |
| Submission Type       | Group report                            |
| Anonymous Group Label |                                         |

**Important anonymisation note:** Do not include student names, student IDs, group ID, email addresses, repository usernames, cloud account details, or any other identifying information in this report. When referring to team members, use Member A, Member B, and Member C. Screenshots, logs, links, filenames, file paths, bucket names, and cloud-console outputs must be checked and anonymised before submission. Source code/repository information is submitted separately through Brightspace and should not be included in this peer-reviewed report if it reveals identity.

### Abstract

This project processes a synthetic cloud service log dataset (50000 records) using Alibaba Cloud OSS for storage, MapReduce for baseline analytics (request counts, server errors, top 10 slow endpoints), and Ray for parallel degraded-service detection. The MapReduce baseline successfully identified service-level request distributions and performance bottlenecks. The Ray extension detected degraded services based on slow request rate ( $>20\%$ ), server error rate ( $>10\%$ ), and timeout count ( $\geq 5$ ). Results show payment-service and search-service as degraded due to high error rates and slow request rates respectively.

### I. Project Overview and Workflow

The synthetic cloud service log dataset contains 50000 records with fields including timestamp, request\_id, service\_name, endpoint, status\_code, response\_time\_ms, and error\_type. The dataset was first uploaded to Alibaba Cloud OSS for persistent cloud storage. Next, MapReduce jobs were implemented to produce three baseline analytics outputs: request count by service, server error count by service (status\_code  $\geq 500$ ), and top 10 slow endpoints (response\_time\_ms  $> 800$ ). Then, a Ray-based extension was developed to detect degraded services by calculating slow request rates, server error rates, and timeout counts in parallel. Finally, the MapReduce batch processing model was compared with Ray's parallel task-based model in terms of implementation approach and execution characteristics.

### II. Dataset and Cloud Object Storage

The dataset was stored in Alibaba Cloud Object Storage Service (OSS) in the Beijing region (oss-cn-beijing). The bucket used is named minilab2-bucket, and the file size is approximately 4.24 MB containing 50000 log records. The dataset was uploaded directly through the OSS web console without any preprocessing or modification.

Object storage is suitable for this log analytics workload because it provides durable, scalable, and cost-effective storage for immutable batch data. Unlike traditional file systems, OSS offers built-in redundancy, accessible via HTTP/HTTPS, and integrates easily with data processing tools. For analytics pipelines, storing raw logs in object storage allows multiple processing frameworks (e.g., MapReduce, Ray) to access the same data without duplicating files.

#### Selected evidence:



### III. MapReduce Baseline Analytics

#### MapReduce explanation:

Take one input record: 2026-04-10T09:15:22Z,R00023,U104,auth-service,/login,POST,200,142,eu-west,

The mapper extracts the service\_name field (index 3) and emits a key-value pair: "auth-service", "1". The reducer receives all values for each key and sums them. For example, all "1"s for auth-service are added to produce the total count: auth-service, 12121.

#### Selected evidence:

```
[root@iZ2zect3m6hilx9vukgemjZ COMP3041J]# python3.11 tools/run_local_mapreduce.py "../Comp3041J MiniProject 2 Dataset.csv" mapreduce/mapper_request_count_by_service.py mapreduce/reducer_sum.py
auth-service      12121
notification-service  8412
order-service     10937
payment-service   7914
search-service    10616
[root@iZ2zect3m6hilx9vukgemjZ COMP3041J]# python3.11 tools/run_local_mapreduce.py "../Comp3041J MiniProject 2 Dataset.csv" mapreduce/mapper_server_error_by_service.py mapreduce/reducer_sum.py
auth-service      436
notification-service  436
order-service     717
payment-service   1362
search-service    904
[root@iZ2zect3m6hilx9vukgemjZ COMP3041J]# python3.11 tools/run_local_mapreduce.py "../Comp3041J MiniProject 2 Dataset.csv" mapreduce/mapper_slow_endpoint_count.py mapreduce/reducer_sum.py --top10
search-service,/search/results  1174
search-service,/search/filter   1165
search-service,/search          1123
search-service,/search/autocomplete  1119
payment-service,/payments       577
payment-service,/payments/refund  526
payment-service,/payments/status  525
payment-service,/payments/confirm 506
order-service,/orders           256
order-service,/orders/history   229
```

### IV. Ray Extension Analytics

The Ray implementation processes the dataset in parallel by splitting it into chunks. Each Ray remote task (partial\_stats) processes one chunk of CSV rows and calculates per-service counters for total requests, slow requests (>800ms), server errors (≥500), and timeout errors. The partial results are then merged using merge\_stats() to produce global statistics. Finally, degraded\_reasons() evaluates each service against three conditions: slow request rate > 20%, server error rate > 10%, or timeout count ≥ 5. Services meeting any condition are output as degraded with the corresponding reason.

#### Required output:

auth-service, repeated timeout errors

notification-service, repeated timeout errors

order-service, repeated timeout errors

payment-service, repeated timeout errors, high server error rate, high slow request rate

search-service, repeated timeout errors, high slow request rate

Degraded services identified:

payment-service: high server error rate (17.2%), high slow request rate, timeout errors  
search-service: high slow request rate (43%), timeout errors

**Selected evidence:**

```
[root@iZ2zect3m6hilx9vukgemjZ COMP3041J_ ]# python3.11 ray_degraded_services.py "../Comp3041J
MiniProject 2 Dataset.csv"
2026-05-14 15:12:03,203 INFO worker.py:2012 -- Started a local Ray instance.
/usr/local/lib64/python3.11/site-packages/ray/_private/worker.py:2051: FutureWarning: Tip: In futur
e versions of Ray, Ray will no longer override accelerator visible devices env var if num_gpus=0 or
num_gpus=None (default). To enable this behavior and turn off this error message, set RAY_ACCEL_EN
V_VAR_OVERRIDE_ON_ZERO=0
  warnings.warn(
auth-service,repeated timeout errors
notification-service,repeated timeout errors
order-service,repeated timeout errors
payment-service,repeated timeout errors
payment-service,high server error rate
payment-service,high slow request rate
search-service,repeated timeout errors
search-service,high slow request rate
```

**V. Validation of Results**

To validate the correctness of the Ray degraded-service detection, I manually calculated the server error rate for payment-service using the MapReduce outputs as ground truth.

From MapReduce Output 1 (request count): Total payment-service requests: 7914

From MapReduce Output 2 (server error count): payment-service server errors (status\_code ≥ 500): 1362

Manual calculation: Error rate = 1362 / 7914 = 0.172

The Ray degradation condition requires "high server error rate" when error rate > 10%. Since 17.2% exceeds this threshold, payment-service should be classified as degraded.

Therefore, ray output "payment-service,high server error rate" is confirmed.

This cross-validation between MapReduce outputs and Ray outputs confirms that both implementations are correct and consistent.

**VI. Bounded Comparison and Analysis**

**Purpose:**

| Item                      | Required answer                                                                                                                                                                             |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MapReduce output evidence | Output 1: auth-service,12121;<br>Output 2: payment-service,1362;<br>Output 3: search-service,/search/results,1174                                                                           |
| Ray output evidence       | payment-service,high server error rate;<br>search-service,high slow request rate;<br>auth-service,repeated timeout errors                                                                   |
| MapReduce logic           | Mapper extracts service_name as key and emits "1" as value; Reducer sums all values per key to produce total request count.                                                                 |
| Ray logic                 | Each Ray task processes a chunk of CSV rows, calculates per-service counters (total, slow, errors, timeout). Results are merged, then conditions (slow_rate>20%, error_rate>10%, timeout≥5) |

|                           |                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                           | determine degraded status.                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Execution evidence</b> | Environment: Alibaba Cloud ECS (2 vCPU, 1.8 GB RAM), Alibaba Cloud Linux 3.2104, Python 3.11.13, Ray 2.55.1<br>MapReduce (Task 1): ~0.5 seconds<br>Ray degraded detection: ~1.2 seconds (including Ray initialization)                                                                                                                                                                                             |
| <b>Conclusion</b>         | MapReduce provides a simple, reliable batch processing model suitable for fixed analytics tasks like counting and aggregation. Ray offers more flexible parallel processing and is better suited for multi-condition detection tasks. For this project, Ray's ability to compute multiple metrics (error rate, slow rate, timeout count) in a single pass was more efficient than running separate MapReduce jobs. |

## VII. Team Integration and Collaboration

Member A: Alibaba Cloud OSS setup, bucket creation, dataset upload, and cloud storage configuration

Member B: MapReduce implementation (3 mappers + reducer) and local runner

Member C: Ray implementation (parallel degraded-service detection) , validation, and report preparation

## VIII. Group-Level Use of GenAI, If Applicable

No group-level GenAI decisions were made. Each team member used GenAI individually for code debugging, error explanation, and report writing assistance as documented in their individual reports. All group-level design decisions like choosing MapReduce key-value pairs, were made collaboratively based on the project requirements and manual code review.